Relay-Bench: Evaluating LLMs on Multi-Domain Reasoning Chains

LIAM SWAYNE
lswayne@andrew.cmu.edu
May 29th, 2026

Abstract. Introducing *Relay-Bench*, an unsaturated, holistic, text-only benchmark that measures LLMs' ability to complete an assortment of tasks from distinct domains in a single prompt. The leading model, GPT-5.5 (xHigh), scores 43.3%. The test set entirely consists of composite problems: groups of single-domain subproblems that are strung together into challenges that require reasoning across multiple domains simultaneously. Many of these problems then have layers of complexity added through prompt encoding and deliberate context bloat. Domains tested include visual reasoning, coding, math, information extraction (with a focus on web search), problem-solving, general knowledge, and data analysis. No restrictions are imposed outside of the model harness, and models are explicitly encouraged to leverage code-execution, web searches, and all available tools. All problems are composed of two to thirteen subproblems and do not require multi-modal input or output.

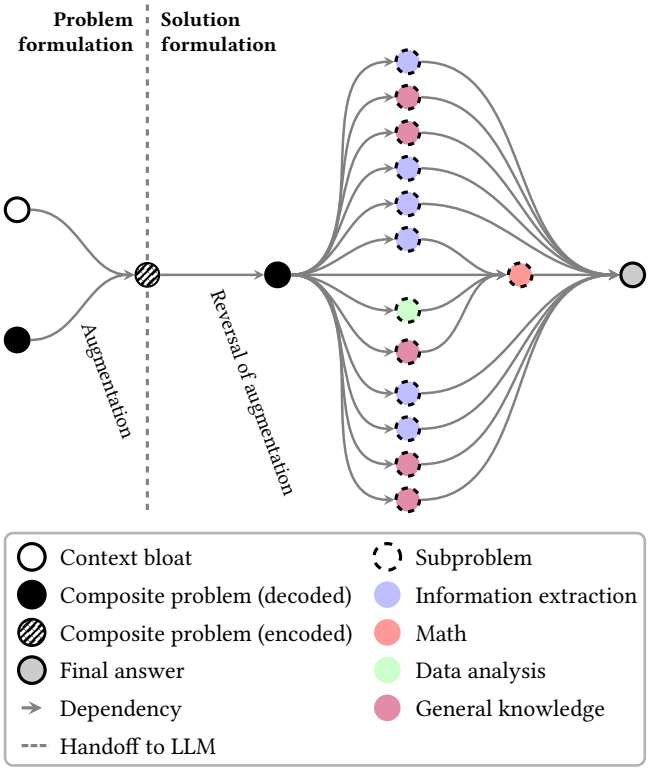

Fig. 1. A composite problem from the private, held-out test set represented as a dependency graph. *Relay-Bench* does not contain cyclic dependencies between subproblems.

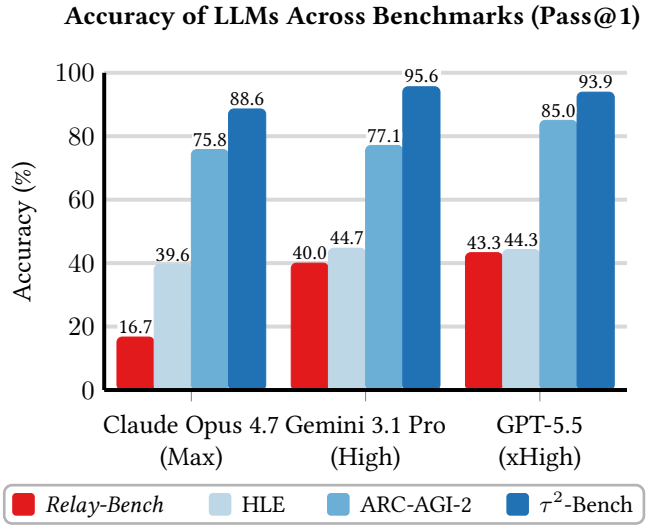

Fig. 2. All models evaluated score lower on *Relay-Bench* than selected widely used benchmarks, including Humanity's Last Exam (HLE), which has not reached saturation more than 15 months after release [1, 2, 3]. GPT-5.5 leads *Relay-Bench* with a score of 43.3%. The average *Relay-Bench* score across the three models tested is 33.3%, compared to averages of 42.9% on HLE, 79.3% on ARC-AGI-2, and 92.7% on τ^2-Bench. The rate of leader progression of comparable benchmarks suggests that *Relay-Bench* could take one to two years to reach saturation.

1 INTRODUCTION

"To do two things at once is to do neither."

—Publilius Syrus

The continual progression of large language models (LLMs) has motivated users and institutions to challenge frontier models [4, 6, 7, 8, 9, 10] with increasingly complex tasks, often in a single prompt [11, 12]. It is not uncommon for a model to have to respond to a prompt that requires reasoning across multiple domains simultaneously, a gap described by Chen et al. as "a critical impediment in the pursuit of artificial general intelligence" [13].

This fact, combined with the mainstreaming of agentic LLM workflows, burgeons demand for models capable of completing numerous operations with high reliability [14]. Many existing benchmarks using a text-in, text-out question-answering format have reached benchmark saturation (defined here as at least one model achieving a Pass@1 score above 90%), and most new LLM benchmarks test individual domains (*e.g.*, GPQA), agentic scenarios (*e.g.*, SWE-bench), file manipulation (*e.g.*, SpreadsheetBench), include questions without a single correct answer (*e.g.*, Chatbot Arena), or use novel scoring frameworks to differentiate models (*e.g.*, τ-bench) [15, 16, 17, 18, 19]. There exists a research gap for an unsaturated, holistic, text-only benchmark capturing LLMs' abilities to answer questions that require reasoning across multiple domains simultaneously.

Frontier models continued to progress throughout 2024 and 2025, with leaders surpassing scores of 90% on GPQA, MATH-500, AIME 2025, and other question-answering benchmarks [20, 21, 22]. Even Humanity's Last Exam, a landmark in scope and difficulty, saw scores rise more than four-fold within a year of the benchmark's release [23]. Throughout this paper, a benchmark's status as *saturated* indicates that at least one model can achieve a Pass@1 score above 90%. Numerous benchmarks reached saturation within two years of their debut, meaningfully lessening their ability to differentiate models, even when aggregated. The ephemeral usefulness of simple, soon-to-be-saturated, question-answering benchmarks may play a part in benchmark creators' pivot towards non-text inputs (*e.g.*, MMMU, MathVista), tool calls (*e.g.*, ToolBench, API-Bank), and non-binary output grading (*e.g.*, MT-Bench, AlpacaEval) [24, 25, 26, 27, 28, 29]. *Relay-Bench* takes an orthogonal approach, strengthening problems by layering complexity within the problems themselves rather than through input modalities and scoring systems.

In designing *Relay-Bench*, a handful of key properties were prioritized:

Reproducibility. Compatibility with new and existing models is maximized by limiting the number of model features required to run an evaluation. Visual input was strategically avoided while drafting visual reasoning problems, yielding a text-in, text-out collection of prompts and answers. Although this constraint may seem to preclude visual reasoning in problems, *Relay-Bench* uses novel techniques to translate visual inputs into text prompts. For example, an ASCII maze is a valid subproblem even though an image of a maze is not. The simple methodology adopted for this benchmark allows for evaluation of preview or beta releases in which modern capabilities—such as tool-calling, forced JSON output, and multi-modal input—are absent. Manually evaluating the capabilities of models through consumer-facing user interfaces is viable due to the simplicity of the testing methodology (pasting a prompt and copying the model's response) and the limited test set size, although this approach is not explored in this paper.

Exactness. The grading framework sidesteps answer classification and LLM judges by providing models with explicit instructions that imply an unambiguous answer. A script deterministically extracts the answer and compares it to the correct answer. GAIA, a benchmark developed by Meta, has similar goals, but uses "quasi exact" answers [30]. For example, GAIA accepts $89706.00 when the correct answer is 89706.00 [30]. *Relay-Bench* problems' prompts include rules an LLM must follow that always narrow the answer down to an exact string, precluding variable answers. For example, each prompt includes granular capitalization and symbol inclusion rules specifying which rules apply to which part of each string. This serves the dual-purpose of making grading simpler and testing models on instruction following abilities.

Simple scoring. Some existing benchmarks report Pass^n success, deflating scores while driving up the cost to evaluate models. This approach was introduced in τ-Bench and adopted by APEX-Agents and LiveMathBench [19, 31, 32]. *Relay-Bench* shifts all score deflation away from the scoring framework and towards the problems themselves, opting for Pass@1 evaluation. Instead of finding a problem that models can solve consistently and using a large n in Pass^n to deflate scores, we merge n independent subproblems with high individual pass rates into a single composite problem that naturally has a low pass rate.

Tool permissive. Features that are ordinarily available when using a consumer-facing interface are often disabled during benchmarking, which may cause benchmark results to diverge from the true capabilities of the model. To create a benchmark that accurately reflects users' experiences with LLMs, all available tools, skills, and other features are enabled when evaluating models on *Relay-Bench*. Models are explicitly informed of their permission to take advantage of these capabilities. Consequently, trivial web search problems were avoided when drafting subproblems for this benchmark. Allowing web search during evaluation introduces a contamination risk, as models may retrieve leaked solutions. This is ordinarily mitigated by using URL blocklists during evaluation [6, p. 244], although such solutions require updating the blocklist periodically to maintain effectiveness. Claude Mythos, a model that lacks a general public release and is considered by some to be the current SOTA LLM [33, 34], was observed locating a benchmark test set and leveraging it to boost its score, demonstrating the risk that public test sets pose to the authenticity of benchmark results [6, p. 36]. *Relay-Bench* opts for a more robust solution: keeping the entire test set private, so no blocklist is required.

Reflective of usefulness for general users. *Relay-Bench* does not attempt to determine the usefulness of LLMs to any individual group of LLM users. Instead, subproblems

are made to mimic real-world use cases across users as a whole. Although subproblems are often at the expert level, they are frequently combined with translation, instruction following, string manipulation, and other generalist tasks. These tasks are not particularly difficult, but their frequency and the fact that they are layered on top of existing challenges serves the purpose of factoring LLM reliability into *Relay-Bench* without including any problems explicitly targeting that area.

2 RELATED WORK

Relay-Bench has similar goals to GAIA, an informally retired benchmark for general AI assistants [30]. Both benchmarks aim to challenge LLMs in generalist roles, but *Relay-Bench* is designed to maximize reproducibility through evaluation simplicity. By contrast, GAIA has a complex evaluation framework, with the authors noting that reproducibility may prove elusive [30].

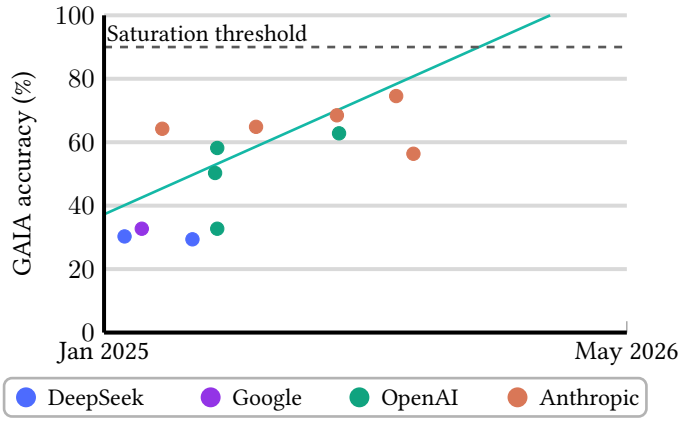

Fig. 3. Each dot represents a model's best GAIA accuracy at release, colored by developer. The line of best fit (turquoise) for record-setting scores on GAIA extrapolates saturation to December 15th, 2025 [35].

3 METHODOLOGY

Every problem in the test set is a *composite problem*, composed of two to thirteen *subproblems*. Each subproblem is akin to a single test set problem in existing question-answering benchmarks (*e.g.*, FrontierMath, LiveCodeBench), and typically targets one domain [36, 37]. Every subproblem can be solved independently, but dependencies are often introduced when combining subproblems. The final answer is always directly or indirectly dependent on all subproblems. For example, a test set problem that requires solving an integral may contain constants that are the answers to other subproblems. No composite problem contains cyclic dependencies between its subproblems (Fig. 1).

Relay-Bench includes 31 problems, 30 of which are kept in a held-out private test set to avoid benchmark contamination [38]. One example problem, found in Appendix A, is not factored into benchmark performance. The private test set can be sharedd with trusted reviewers under non-disclosure.[1]

Anthropic [39], Google AI Studio [40], and OpenAI APIs [41] were used to evaluate LLMs on composite problems. All tools are enabled, the prompt is sent, the response JSON is parsed, and the answer is scored. Thinking effort is set to the maximum setting for all the models evaluated, which is "Max" for Claude Opus 4.7 [42], "High" for Gemini 3.1 Pro [43], and "xHigh" for GPT-5.5 [44].

3.1 Question generation

Relay-Bench incorporates both human-written and generated subproblems. The process of generating subproblems starts with identifying a class of challenges that is narrower than a domain, containing near-identical problems with different data plugged in. For example, there are multiple sudoku subproblems, each containing different sudoku games. A golden example is written by a human, and the subproblem set is proliferated by generating similar instances of the same problem using Claude Opus 4.7, Gemini 3.1 Pro, GPT-5.4 Mini, and Claude Sonnet 4.6. Generated subproblems are reviewed for accuracy by a human, similar to the question generation process used in AA-Omniscience [45]. Two subproblems of the same class do not challenge models in distinct ways, so only one subproblem from a class can appear in the each composite problem.

Model performance was measured periodically throughout the creation of the benchmark to determine which problems required revision to differentiate model performance. Composite problems that did not cause model failure were further augmented with additional subproblems or subsumed into larger composite problems.

Relay-Bench exhausts LLMs with sequential tasks of grand scope, tempting models to overlook crucial details in individually easy subproblems. For example, a subproblem asking how many days a deceased, well-known figure lived is trivial at its core, yet invites failure when a model neglects to account for leap years.

3.2 Prompt Encoding

A nonce prompt encoding system is leveraged to add an additional layer of difficulty and a source of "exhaustion" for models to handle. A composite problem that is already well-formed is pasted into a translator that adds 7813 characters of fabricated "user preferences" that are entirely irrelevant to the problem. The results of Du et al. indicate that context bloat degrades the performance of LLMs, and *Relay-Bench*

[1] All three models evaluated in this paper were tested on the problem. Only GPT-5.5 answered correctly.

utilizes this technique to make test set problems more challenging and reflective of real use-cases [46]. Secondly, each word, symbol, or number in the prompt is substituted with a three-letter alphabetic string enclosed in angle brackets. For example "bottle" may translate to "<AvY>". These translations are case sensitive, and the encoder intentionally reuses the same string with different capitalization to attempt to get models to confuse two unique symbols. In the encoded prompt the model is provided with a brief explanation of how to decode the prompt, and then instructed to do as the prompt asks after decoding it. The model is then provided a dictionary of translations, followed by the encoded prompt. Some of the longest *Relay-Bench* problems require executing more than 10,000 translation operations to extract the composite problem prompt. The prompt encoding process is entirely automated, and applied identically to a subset of the problems in the test set. An encoded form of the public example problem is in Appendix B.

4 RESULTS

4.1 Performance and Error Analysis

GPT-5.5 and Gemini 3.1 Pro exhibit approximately equivalent performance on *Relay-Bench* and Humanity's Last Exam, while Claude Opus 4.7's score on *Relay-Bench* is less than half of its score on Humanity's Last Exam [3].

Only 50 of the 90 responses models gave contained a parseable final answer (Fig. 4).

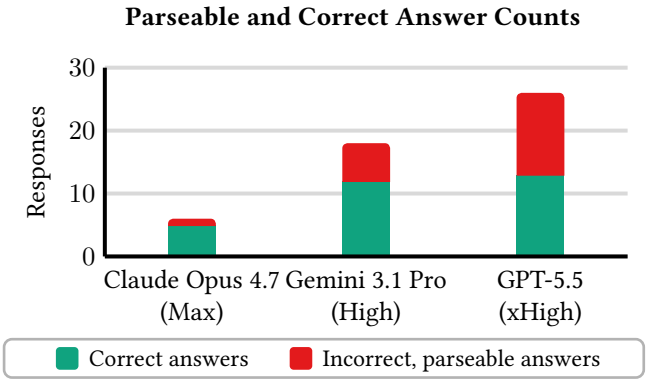

Fig. 4. Counts of correct and incorrect parseable responses. The total height of each bar is the number of parseable responses returned by a model.

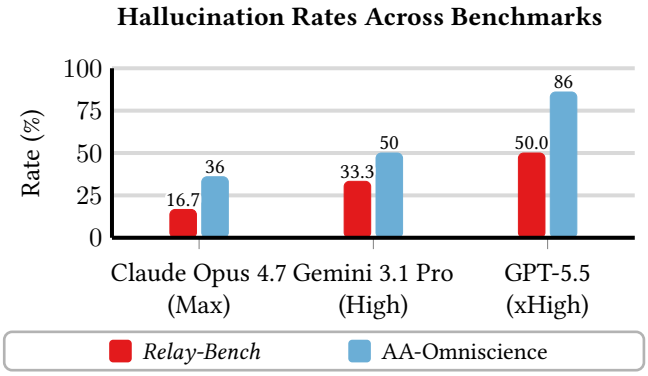

Fig. 5. The fraction of parseable responses that were incorrect on *Relay-Bench* and the AA-Omniscience hallucination-rate evaluation [47].

Claude Opus 4.7 returned the fewest parseable answers, but had the lowest hallucination rate because only one of them was incorrect (Fig. 4, Fig. 5). *Relay-Bench* provides no incentive to abstain from answering, suggesting that Claude Opus 4.7 has a greater internal incentive to avoid hallucinating than Gemini 3.1 Pro and GPT-5.5. These results align with hallucination rates for the three models on AA-Omniscience [47]. The penalty for not responding is equivalent to the penalty for an incorrect answer, encouraging models to hallucinate and guess rather than abstaining from answering [49]. Despite being presented with the same incentives, the three models tested had greatly differing hallucination rates (Fig. 5). There are several instances of models being unable to solve a specific subproblem, guessing, and submitting their composite problem answer. There are no cases in which models stopped attempting a composite problem after being unable to solve a subproblem.

Seven of the problems in the test set are longer than 15,000 characters. Gemini 3.1 Pro and GPT-5.5 solved one of seven, and Claude Opus 4.7 solved none, providing evidence that this class of problems is significantly more difficult than others. Four of these problems are encoded (section 3.2), which adds thousands of characters to the problem. Claude Opus 4.7 explicitly halted its response to every encoded problem in this subset (JSON responses from Claude Opus 4.7 included `stop_reason=refusal`), suggesting that the Claude Mythos safeguards Anthropic is testing through Claude Opus 4.7 are overly-conservative [33].

4.2 Cost

Including costs for test runs and question generation, the cost of developing this benchmark and running it on the models tested was $163.29. Claude Opus 4.7 was the most expensive model to evaluate, costing $33.76, an average of $1.09 per problem. Gemini 3.1 charges the lowest input, cached input, and output token rates by a significant margin, but those savings are entirely lost by a remarkably low

input cache rate of 0.6% compared to Claude Opus 4.7 at 95.0% and GPT-5.5 at 6.5%. Interestingly, the cache rates of the three models are all at least an order of magnitude apart. The cheapest model of the three was GPT-5.5, a 32% discount compared to Gemini 3.1 Pro.

	Claude Opus 4.7 (Max)	**Gemini 3.1 Pro (High)**	**GPT-5.5 (xHigh)**
Input token price ($/M)	$5.00	$2.00	$5.00
Input token usage	1,665,250	12,902,461	2,253,346
Input cost	$8.33	$25.80	$11.27
Cached input token price ($/M)	$0.50	$0.20	$0.50
Cached input token usage	31,794,679	76,623	157,568
Cached input cost	$15.90	$0.02	$0.08
Output token price ($/M)	$25.00	$12.00	$30.00
Output token usage	381,340	597,718	369,782
Output cost	$9.53	$7.17	$11.09
Total cost	**$33.76**	**$32.99**	**$22.44**

Table 1. A breakdown of the costs associated with running each model. The cost of code execution containers instantiated by models was negligible and omitted.

Notably, Claude Opus 4.7's input token usage (including cached tokens) summed across internal tool call iterations exceeded two million on six problems, despite all *Relay-Bench* prompts being under 50,000 tokens. Extensive chains of thought and tool use compounded to grow input costs. Even with caching enabled, input tokens constituted the majority of the costs associated with running the benchmark. One Claude Opus 4.7 run used 3.7 million input tokens across 41 tool calls; three internal iterations accounted for approximately one million input tokens each, driven by lengthy web search and code execution results.

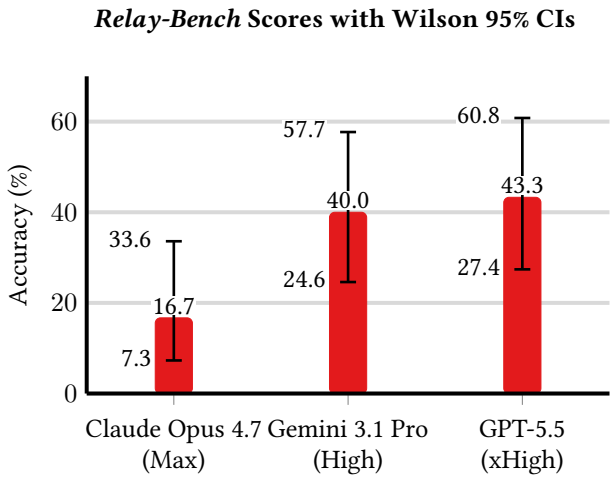

Fig. 6. Wilson 95% confidence intervals of *Relay-Bench* scores when evaluated with Pass@1. Due to the small test set size of 30, the confidence intervals are far from ideal. In particular, the range of Claude Opus 4.7's confidence interval is greater than the score itself. By comparison, Humanity's Last Exam has 2500 problems [50] and GAIA has 466 problems [30].

5 LIMITATIONS AND FUTURE WORK

Relay-Bench is held back due to resource, time, and harness constraints. Many existing models, even with reasoning effort set to the maximum setting, simply terminate before returning an answer or hit a tool call limit. A more robust harness could force models to continue attempting to solve a problem even if a limit is hit. Although this benchmark is designed with breadth of models in mind, it was only run on three models due to cost constraints. The models selected for this evaluation were chosen because they are widely accepted to be the leading publicly released models prior to the time of writing[2]. Evaluating more models on this benchmark may reveal additional insights into model capabilities. In particular, investigating a potential correlation between the parameter-counts of open-source models and their performance may yield insight into the relation between LLMs' robustness to "exhaustion". Exploring models without chain of thought, particularly those with reasoning counterparts, may reveal the size, or lack thereof, of the gap in long-reasoning capabilities between standard LLMs and reasoning models. Testing through site UIs may also prove valuable in determining the gap in abilities between models' consumer-facing offerings and their API counterparts.

Many *Relay-Bench* problems are unsolvable without web search or internalization of esoteric knowledge extracted

[2]Some speculate that Claude Mythos is the current SOTA LLM [33], but it is not publicly released. Claude Opus 4.8 was released during the writing of this paper (after the models tested in this paper had been evaluated) [5], and is considered the current SOTA LLM, occupying the top spot on the Artificial Analysis Intelligence Index leaderboard [48].

from the web. Additionally, some subproblems are impractical to solve without code-execution (*e.g.*, solving ten sudoku games). Search problems intentionally rely on ground truth sources that are unlikely to change. However, if an answer becomes unavailable, it requires replacement, preventing *Relay-Bench* from being maintenance free indefinitely. In the future, a similar benchmark narrowing the scope of capabilities required to reach a solution, particularly one that removes search and code execution subproblems, may be a better measure of LLM capabilities. Alternatively, code execution and search tools could be sandboxed, although such restrictions would sacrifice the simplicity and reproducibility of the evaluation framework. These restrictions were not applied to *Relay-Bench* to avoid reducing the already small problem set. Models' low scores on *Relay-Bench* combined with the small problem count resulted in large confidence intervals for model performance on this benchmark (Fig. 6).

Some subproblems in the test set require either knowledge of—or the ability to search for and process—documents within a corpus where the correct document meets a set of criteria that are non-trivial to evaluate. Solving these subproblems may require a new approach to managing context windows given that frontier LLMs cannot fit all the documents in the search space in their current context windows, which are presently limited to around one million tokens. These problems would be especially time-consuming for humans attempting to solve them compared to the rest of the test set. It is a common practice to compare LLMs to human panels, but doing so is neither essential to differentiate frontier models nor achievable given the limited budget and scope of this research. However, evaluating humans on *Relay-Bench* problems would provide insight into the duration it takes humans to solve these problems, and by extension determine how much of a human's time can be replicated with a one-shot request to an LLM.

Based on the limitations of *Relay-Bench* the prime concerns of an effort to create a sharpened revision that supplants the original should be growing the problem set and replacing existing problems with substitutes requiring a narrower set of capabilities to reach a solution.

REFERENCES

[1] ARC Prize. *ARC Prize Leaderboard.* https://arcprize.org/leaderboard. Accessed: May 22, 2026.

[2] Artificial Analysis. *τ^2-Bench Telecom Benchmark Leaderboard.* https://artificialanalysis.ai/evaluations/tau2-bench?models=gpt-5-5%2Cgemini-3-1-pro-preview%2Cclaude-opus-4-7. Accessed: May 22, 2026.

[3] Artificial Analysis. *Humanity's Last Exam Benchmark Leaderboard.* https://artificialanalysis.ai/evaluations/humanitys-last-exam?models=gpt-5-5%2Cgemini-3-1-pro-preview%2Cclaude-opus-4-7. Accessed: May 22, 2026.

[4] Anthropic. *Introducing Claude Opus 4.7.* Apr. 16, 2026. https://www.anthropic.com/news/claude-opus-4-7.

[5] Anthropic. *Introducing Claude Opus 4.8.* May 28, 2026. https://www.anthropic.com/news/claude-opus-4-8.

[6] Anthropic. *Claude Mythos Preview System Card.* 2026. https://www-cdn.anthropic.com/08ab9158070959f88f296514c21b7facce6f52bc.pdf. Accessed: May 27, 2026.

[7] The Gemini Team, Google. *Gemini 3.1 Pro: A smarter model for your most complex tasks.* Feb. 19, 2026. https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/.

[8] OpenAI. *Introducing GPT-5.5.* Apr. 23, 2026. https://openai.com/index/introducing-gpt-5-5/.

[9] xAI. *Grok 4.3 Beta.* Apr. 17, 2026. https://grok.com/release-notes/apr-17-2026.

[10] Qwen Team, Alibaba Cloud. *Qwen3.7: The Agent Frontier.* May 19, 2026. https://qwen.ai/blog?id=qwen3.7.

[11] Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. *From Crowdsourced Data to High-Quality Benchmarks: Arena-Hard and BenchBuilder Pipeline.* arXiv:2406.11939, 2024. https://arxiv.org/abs/2406.11939.

[12] Bill Yuchen Lin et al. *WildBench: Benchmarking LLMs with Challenging Tasks from Real Users in the Wild.* arXiv:2406.04770, 2024. https://arxiv.org/abs/2406.04770.

[13] Xuanzhong Chen et al. *AgentFrontier: Expanding the Capability Frontier of LLM Agents with ZPD-Guided Data Synthesis.* arXiv:2510.24695, 2025. https://arxiv.org/abs/2510.24695.

[14] Melissa Z. Pan et al. *Measuring Agents in Production.* arXiv:2512.04123, 2025. https://arxiv.org/abs/2512.04123.

[15] David Rein et al. *GPQA: A Graduate-Level Google-Proof Q&A Benchmark.* arXiv:2311.12022, 2023. https://arxiv.org/abs/2311.12022.

[16] Carlos E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* arXiv:2310.06770, 2023. https://arxiv.org/abs/2310.06770.

[17] Zeyao Ma et al. *SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation.* arXiv:2406.14991, 2024. https://arxiv.org/abs/2406.14991.

[18] Wei-Lin Chiang et al. *Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference.* arXiv:2403.04132, 2024. https://arxiv.org/abs/2403.04132.

[19] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.* arXiv:2406.12045, 2024. https://arxiv.org/abs/2406.12045.

[20] Artificial Analysis. *GPQA Diamond Benchmark Leaderboard.* https://artificialanalysis.ai/evaluations/gpqa-diamond. Accessed: May 23, 2026.

[21] Artificial Analysis. *MATH-500 Benchmark Leaderboard.* https://artificialanalysis.ai/evaluations/math-500. Accessed: May 23, 2026.

[22] Artificial Analysis. *AIME 2025 Benchmark Leaderboard*. https://artificialanalysis.ai/evaluations/aime-2025. Accessed: May 23, 2026.

[23] Center for AI Safety, Scale AI, and HLE Contributors Consortium. *A benchmark of expert-level academic questions to assess AI capabilities*. Nature 649, 1139–1146 (2026). https://arxiv.org/abs/2501.14249.

[24] Xiang Yue et al. *MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI*. CVPR, 2024. https://arxiv.org/abs/2311.16502.

[25] Pan Lu et al. *MathVista: Evaluating Mathematical Reasoning of Foundation Models in Visual Contexts*. ICLR, 2024. https://arxiv.org/abs/2310.02255.

[26] Yujia Qin et al. *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs*. arXiv:2307.16789, 2023. https://arxiv.org/abs/2307.16789.

[27] Minghao Li et al. *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*. arXiv:2304.08244, 2023. https://arxiv.org/abs/2304.08244.

[28] Lianmin Zheng et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. arXiv:2306.05685, 2023. https://arxiv.org/abs/2306.05685.

[29] Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. *Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators*. arXiv:2404.04475, 2024. https://arxiv.org/abs/2404.04475.

[30] Grégoire Mialon et al. *GAIA: a benchmark for General AI Assistants*. arXiv:2311.12983, 2023. https://arxiv.org/abs/2311.12983.

[31] Bertie Vidgen et al. *APEX-Agents*. arXiv:2601.14242, 2026. https://arxiv.org/abs/2601.14242.

[32] Junnan Liu et al. *Are Your LLMs Capable of Stable Reasoning?* arXiv:2412.13147, 2024. https://arxiv.org/abs/2412.13147.

[33] Anthropic. *Project Glasswing*. 2026. https://www.anthropic.com/project/glasswing. Accessed: May 26, 2026.

[34] Claude Fast. *Claude Mythos Preview: Anthropic's Frontier Model Explained*. 2026. https://claudefa.st/blog/models/claude-mythos. Accessed: May 26, 2026.

[35] Princeton Language and Intelligence. *Holistic Agent Leaderboard: GAIA*. https://hal.cs.princeton.edu/gaia. Accessed: May 26, 2026.

[36] Elliot Glazer, Ege Erdil, Tamay Besiroglu, et al. *FrontierMath: A Benchmark for Evaluating Advanced Mathematical Reasoning in AI*. arXiv:2411.04872, 2024. https://arxiv.org/abs/2411.04872.

[37] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code*. arXiv:2403.07974, 2024. https://arxiv.org/abs/2403.07974.

[38] Chunyuan Deng et al. *Investigating Data Contamination in Modern Benchmarks for Large Language Models*. arXiv:2311.09783, 2024. https://arxiv.org/abs/2311.09783.

[39] Anthropic. *Anthropic API documentation*. https://docs.anthropic.com/en/api/overview. Accessed: May 27, 2026.

[40] Google. *Gemini API documentation*. https://ai.google.dev/gemini-api/docs. Accessed: May 27, 2026.

[41] OpenAI. *OpenAI API reference*. https://platform.openai.com/docs/api-reference. Accessed: May 27, 2026.

[42] Anthropic. *Adaptive thinking*. https://platform.claude.com/docs/en/build-with-claude/adaptive-thinking. Accessed: May 27, 2026.

[43] Google. *Gemini thinking*. https://ai.google.dev/gemini-api/docs/thinking. Accessed: May 27, 2026.

[44] OpenAI. *Reasoning models*. https://developers.openai.com/api/docs/guides/reasoning. Accessed: May 27, 2026.

[45] Declan Jackson, William Keating, George Cameron, and Micah Hill-Smith. *AA-Omniscience: Evaluating Cross-Domain Knowledge Reliability in Large Language Models*. arXiv:2511.13029, 2025. https://arxiv.org/abs/2511.13029.

[46] Yufeng Du et al. *Context Length Alone Hurts LLM Performance Despite Perfect Retrieval*. arXiv:2510.05381, 2025. https://arxiv.org/abs/2510.05381.

[47] Artificial Analysis. *AA-Omniscience: Knowledge and Hallucination Benchmark*. https://artificialanalysis.ai/evaluations/omniscience?models=gemini-3-1-pro-preview%2Cclaude-opus-4-7%2Cgpt-5-5#omniscience-hallucination-rate-tabs. Accessed: May 29, 2026.

[48] Artificial Analysis. *Artificial Analysis Intelligence Index*. https://artificialanalysis.ai/evaluations/artificial-analysis-intelligence-index. Accessed: May 29, 2026.

[49] A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang. *Why Language Models Hallucinate*. arXiv:2509.04664, 2025. https://arxiv.org/abs/2509.04664.

[50] Long Phan et al. *Humanity's Last Exam*. arXiv:2501.14249, 2025. https://arxiv.org/abs/2501.14249.

APPENDIX

A / PUBLIC EXAMPLE PROBLEM

```
A benchmark for evaluating language models on mathematics competition problems, MathArena, has a paper
associated with it. The paper contains at least one, but possibly multiple ranking tables. One table
appears directly above a performance graph that plots results for o4-mini and Qwen3-30B-A3B, among other
models.

In that specific table, each row contains a label followed by additional information in parentheses.
What is the string that appears immediately to the left of the first opening parenthesis in the 4th row
of that table? Trim any leading or trailing spaces from your answer. Put your answer in all caps and
replace any spaces with underscores.

Let A_STR be that string.

Example: ”GEMINI_2.0_FLASH”

In graph theory, the local complementation of a graph G at a vertex v is the operation that toggles every
edge between distinct neighbors of v, replacing the induced subgraph on the open neighborhood N(v) with
its complement, while leaving all edges not between two neighbors of v unchanged.

A graph H is a vertex-minor of a graph G if H can be obtained from G by first taking an induced subgraph
and then applying a finite sequence of local complementations.

For a positive integer k, define f(k) as the smallest integer n such that every simple graph on exactly
n vertices contains the edgeless graph on k vertices (k pairwise non-adjacent vertices with no edges
among them) as a vertex-minor.

It is known that f(k) = 2^k - 1 for k = 1, 2, and 3.

What is f(4)? Let B_STR be that integer.

Example: ”7”

In a single bar of a song by Kendrick Lamar, the names of two different real people are merged together
into one composite name. Identify both people and return their full legal names (including all given
names and any middle names). List both people's full legal names in alphabetical order by last name,
separated by a single underscore. Use an underscore anywhere you would use a space in their names. Put
your answer in all caps.

Let C_STR be that string.

Example: ”ABIGAIL_MAY_JONES_ZACHARY_BLAKE_SMITH”

What woman is implied to be a part of a Kendrick album, exists as a concept rather than a literal person,
and uses a gun? Put your answer in all caps. Replace any spaces with underscores.

Let D_STR be that string.

Example: ”SCARLET_WITCH”

A slot-table is a data structure with n slots supporting three operations:

- Allocate(key) -> ref: Claims a free slot for `key` and returns a compressed reference `ref`.
- Dereference(key, ref) -> slot_index: Returns the index of the slot claimed by `key`. Only valid when
called with the same key that produced `ref`.
- Free(key, ref): Releases the slot.
```

Because `ref` is only meaningful together with its owner `key`, the pair (key, ref) encodes the slot index rather than `ref` alone. This allows `ref` to use far fewer bits than the log(n)-bit lower bound that would apply to a standard standalone pointer into an array of size n.

The table operates at load factor 1 - 1/k, meaning at most n(1 - 1/k) slots can be simultaneously occupied.

Fixed-size references require every `ref` to use the same number of bits. A 2021 theoretical computer science result established tight asymptotic bounds -- a matching construction and a lower bound -- on the minimum achievable fixed-size reference length.

What is the asymptotically tight bound in Theta-notation on the optimal fixed-size reference size in bits, as a function of n and k?

Answer format rules for E_STR (follow exactly or your answer will not match):
- Use the ASCII uppercase letter O (not ”Theta” or ”Big-Theta”)
- Every log must have parentheses around its argument: log(x), never log x
- You do not need to specify the base of any log
- Exponents use the form x^(y) -- parentheses are non-optional
- If a term contains both a variable and a log of that variable, the variable goes to the left of the log: xlog(x), not log(x)x
- If your answer has multiple additive terms, order them left to right from most characters to fewest characters
- Spaces to the left and right of + and - symbols

Let E_STR be the Theta expression exactly as written.

Example: ”O(nlog(n) + n^(2))”

The Alliant Cashback Visa Signature offered 2.5% flat-rate cashback on purchases. What is the exact date that the discontinuation of this perk took effect? Format your answer as YYYYMMDD with no dashes or other separators.

Let F_STR be that date string.

Example: ”20240101”

Fill in the blanks:

_ _____ ____ __________

Each underscore represents one letter and spaces separate words. The answer is four words with 1, 5, 4, and 10 letters respectively.

The answer is something related to all of the following: cult, gun, YouTube, grapple hook, island, MatPat, claymation, zombies.

Put your answer in all caps and replace any spaces with underscores.

Let G_STR be that string.

Example: ”THE_POWER_OF_FRIENDSHIP”

As an LLM, you are allowed to use any tools, skills, or other features at your disposal to solve this problem.

Put your answer in a code block like this:

```
A_STR-B_STR-C_STR-D_STR-E_STR-F_STR-G_STR
```

Example: ”GEMINI_2.0_FLASH-723-ABIGAIL_MAY_JONES_ZACHARY_BLAKE_SMITH-SCARLET_WITCH-O(nlog(n) + n^(2))-20240101-THE_POWER_OF_FRIENDSHIP”

For string variables A, C, D, and G: any non all-caps letter should be replaced with an underscore. Include B_STR, E_STR, and F_STR exactly as computed with no modification.

Throughout this problem when using big-O, it means big-theta. No non-ASCII characters should appear in your answer.

Do not include any other code blocks in your response.

B / PUBLIC EXAMPLE PROBLEM (ENCODED FORM)

```
<user_context>
  <preferences>
      `/uc` anywhere in prompt = ultra-concise mode. Respond with the full answer and zero
      opening/closing remarks, no unnecessary words. Examples: integral → ”127”. Auction list → bare
      bullets, no preamble. All reasoning, verification, and explanation go in `<thinking>` blocks only
      — never in the response. Think extensively in `<thinking>` to ensure correctness before the
      ultra-concise reply.

      BAD /uc response example (never do this): Q: ”which auctions not strategy proof /uc” BAD A: ”From
      the slides, mechanisms not strategy-proof: - English auction – strategy-proof only under private
      values - First-price sealed-bid – bidders shade bids - Dutch auction – equivalent to first-price”
      Wrong because: has preamble, inline explanations, conversational framing. GOOD A: ”- First-price
      sealed-bid\n- Dutch auction”
  </preferences>
  <user_summary>
      **Work context**

      Ethan Marlowe is a student at a top-15 technical university in the northeastern US with part-time
      self-employment income from software contracting work. He is also working as a trainer/reviewer on
      an AI image evaluation and ranking project for a tech client.

      **Personal context**

      Ethan has interests in competitive programming, DC and indie comics, music (hip-hop, indie folk,
      art rock, hyperpop), and investing. He is part of a student engineering team at his university,
      involved in both the technical and documentation side of vehicle design. He collects character
      plushies and has interest in personal web aesthetics (retro web buttons, personal static hosting).
      Ethan uses a desktop all-in-one machine with a SoC (system on a chip) architecture.

      **Top of mind**

      Ethan is working on his engineering team's annual race event, including race time estimation across
      multiple competing teams using timing data. His team's current vehicle documentation involves
      technical copywriting and material science details including thermal properties of polycarbonate
      filament and high-strength aluminum alloy use.

      **Brief history**
```

Recent months

Ethan has been deeply involved in a university competitive programming competition (launched mid-March), building a Python bot with a sense-think-act architecture and goal-based voting system. Key work included a full vision radius sense module, conveyor chain logic, harvester placement, enemy infrastructure destruction via self-destruct, and fixing a series of import and movement bugs.

Ethan completed his prior-year tax filings across federal, two state returns, and a local return (mailed with check). His home address is in a township outside a major city, meaning no local income tax obligation from the city itself. He filed a local return in his university city at a ~3% EIT rate for part-year residency and self-employment income.

He opened a new checking account for a sign-up bonus and is pursuing multiple bank account bonuses simultaneously. He also opened a Roth IRA and a rewards credit card during a session exploring financial aid interactions with retirement accounts, and is building a stock portfolio with holdings including large-cap tech and fintech. His investment thesis centers on robust structural monopolies with consistent revenue growth.

Ethan is building and maintaining a personal infrastructure system: a cloud object storage bucket, a serverless worker, and a GitHub repo for his personal note vault. He has an HTML upload tool for uploading encrypted audio and images, with fixes for binary corruption, special character handling, and web archiving. He also runs a shell script to batch-archive files to the Wayback Machine.

Ethan is managing postural issues with a flexibility routine and has an active skincare routine involving multiple prescription and OTC products. He is on a dermatologist-prescribed treatment course affecting shaving and product choices.

He took a linguistics exam covering phonetics, phonology, morphology, and syntax, and completed a homework assignment for a game theory/social choice course. He also studied bigram language models and Laplace smoothing for an applied machine learning quiz.

Earlier context

Ethan registered to vote in his university's city at his campus-adjacent address. He filed quarterly estimated taxes as a self-employed contractor. He explored credit card strategy, cashback optimization, and bank account bonuses. He researched leveraged index investing, P/E ratio analysis, and major AI lab funding structures.

He developed an IPA drag-and-drop study app (self-contained HTML, dark academic aesthetic) and an interval timer HTML tool. He built and debugged a CSS snippet for link color customization in his note-taking app.

Long-term background

Ethan has sustained interests in AI model architecture, competitive programming infrastructure, personal finance optimization, and comic books. He has explored retro web aesthetics, local AI model execution, and has ongoing familiarity with serverless workers, GitHub Actions, and cloud sync tooling.

</user_summary>
<recent_changes>
- Music preference: prefers concept albums where the artist plays a character and tells a story; especially values interludes and skits that contribute to worldbuilding over conventional songs.
- Emoji display requests: use visualize:show_widget (never a file). Show max relevant emojis as cards (64px emoji, label, Copy button). Copy btn: always black via inline btn.style.cssText with !important on all states. On copy: color:#4ade80 ”Copied!” for 1.5s. No hover bg changes. Cards: var(--bg-200) bg, var(--border-100) border, 14px radius.
- User is a university student, born 2005.

```
      - User skills live on the user's machine at Desktop/Notes/Agent Skills/<skill-name>/SKILL.md. This
      is the single source of truth.
      - Skill edit workflow: (1) read SKILL.md, (2) edit for diffs, (3) copy SKILL.md AND every file in
      references/, (4) package into a zip, (5) present the .skill zip.
      - New skill workflow: create directory, write blank SKILL.md, paste full content, then sync to
      assistant environment via copy + present_files. Never write full content from assistant's side.
      - When asked to create a bookmark: create a .url file with content
      `[InternetShortcut]\nURL=notes://web-open?url=SITE_URL`, filename = bookmark title.
      - The @ folder stores two .url file types: bookmarks (notes://web-open?url=...) and aliases
      (notes://open?vault=Notes&file=NOTE_NAME).
      - For comparisons, prefer a table. When /uc is active, especially favor a table if possible.
      - When user references editing files without specifying a path, it's always on the Desktop or in
      the notes vault directory.
      - For carbon offset prices, always search online for current market rates. Never give a made-up
      range — cite a real source.
      - For diagrams: never use the visualize tool. Use an SVG artifact instead to reduce token usage.
      - Addresses: university address — 400 W. Garrison Ave, Apt 214, [City], [State] [ZIP]. Home address
      — 3318 Fenwick Ct, [City], [State] [ZIP]. Mailing address — student mailbox on campus.
  </recent_changes>
  <summary_of_last_chat>
      User was debugging a flaky CSS animation on his personal note vault landing page — a marquee of
      retro web buttons that would occasionally desync after the tab regained focus. We narrowed it down
      to a stale requestAnimationFrame handle held across visibilitychange events, swapped in a CSS-only
      keyframe approach for the steady-state loop, and kept JS only for the pause-on-hover behavior. He
      then asked for a small SVG divider in the same 88x31 button aesthetic to sit above the marquee.
  </summary_of_last_chat>
</user_context>

You are being challenged for a benchmark. The rest of this prompt has been encoded. Use the dictionary
below to decode the prompt. Do not encode your final answer to the prompt below.

```
{
  <rCL> : neighbors
  <yWI> : POWER
  <Djk> : Claims
  <sPb> : distinct
  <ArI> : performance
  <eKu> : separators
  <avo> : last
  <OHv> : but
  <gOg> : caps
  <uVe> : number
  <psp> : has
  <xSZ> : from
  <SMb> : occupied
  <zcW> : followed
  <gVD> : be
  <cyr> : Spaces
  <axw> : specify
  <EKS> : mathematics
  <XMh> : standalone
  <pbv> : complement
  <Zmp> : result
  <ExV> : no
  <JJz> : leaving
  <mWT> : exactly
  <khZ> : two
```
```

```
<phr> : expression
<Efc> : exact
<Asr> : fewest
<KkM> : respectively
<iHw> : took
<nsB> : hook
<GEv> : fewer
<TfK> : Each
<AFn> : blanks
<BCG> : You
<ZrT> : owner
<oHp> : together
<ndx> : MathArena
<NGH> : List
<LHZ> : term
<Vnv> : asymptotically
<PgQ> : Visa
<iJY> : implied
<xiW> : notation
<uVw> : it
<QwG> : What
<EaR> : merged
<gxz> : theta
<yTY> : features
<Tek> : most
<JuZ> : standard
<lYD> : ?
<YXK> : >
<XEE> : Fill
<WYc> : Replace
<sqL> : s
<ACh> : operates
<IZD> : FLASH
<gmj> : free
<oWc> : structure
<gzB> : among
<vMk> : skills
<uKX> : smallest
<oPz> : row
<RNF> : load
<mds> : No
<oIO> : known
<eOo> : ^
<prh> : mini
<dVt> : integer
<OpO> : argument
<LQz> : by
<BmK> : MAY
<IrP> : following
<gxy> : -
<BGZ> : finite
<FJU> : allows
<yBr> : G
<rKT> : matching
<VPu> : spaces
<LLc> : results
<cwu> : code
<pRz> : THE
```

```
<Qgf> : left
<NYA> : to
<NDA> : Use
<pPL> : operation
<KNh> : need
<eNa> : 3
<tiT> : 1
<cIt> : names
<Qvu> : Answer
<KkX> : If
<tEy> : offered
<Kfk> : name
<FuI> : vertices
<TyO> : of
<aXy> : including
<xWC> : anywhere
<SxB> : not
<FVQ> : MatPat
<DJz> : can
<fUP> : In
<MkU> : and
<JVi> : other
<TPx> : +
<NuI> : one
<NAO> : Returns
<VBP> : four
<nsH> : .
<NtB> : graph
<GyP> : literal
<ONV> : induced
<blh> : complementation
<jLN> : bar
<yfW> : theory
<HSI> : %
<Nnb> : Do
<pmf> : Put
<jfp> : when
<RAy> : complementations
<SXl> : response
<oYW> : single
<pGz> : with
<QXS> : specific
<dQN> : Only
<iID> : bits
<loP> : label
<Kgy> : would
<FJH> : big
<HSs> : underscores
<WUk> : Dereference
<gxb> : size
<vSu> : GEMINI
<bcz> : '
<Cbr> : disposal
<DuY> : modification
<vMH> : into
<DVW> : WITCH
<Gea> : such
<tqN> : then
```

```
<ZRs> : means
<pQB> : solve
<KFi> : benchmark
<PXQ> : its
<EyO> : are
<UbW> : data
<eVK> : lower
<PzH> : additional
<vta> : neighborhood
<RCG> : is
<MhQ> : any
<RHy> : Signature
<vBt> : Unicode
<MWr> : leading
<pUM> : use
<APy> : YouTube
<NCG> : edgeless
<KdF> : reference
<KQJ> : non
<uMR> : or
<HKo> : xlog
<JYw> : log
<BEI> : full
<MAv> : k
<pzd> : Example
<mRY> : index
<Exu> : local
<qyv> : your
<kzX> : claimed
<ZEv> : replacing
<zaL> : =
<Wny> : taking
<YZh> : order
<gUq> : should
<mIH> : each
<TtM> : slot
<aaa> : words
<MOA> : apply
<sAJ> : N
<qkv> : Cashback
<AgK> : subgraph
<Cjx> : something
<UyH> : key
<iQH> : toggles
<ICk> : parentheses
<jOr> : ref
<uol> : pairwise
<lrV> : Throughout
<rlx> : in
<wAY> : legal
<FsZ> : the
<IVg> : simple
<Ydj> : between
<cMt> : minimum
<SqB> : dashes
<wPo> : o
<Let> : string
<bRh> : island
```

```
<QXh> : Allocate
<Bye> : This
<HLO> : three
<gba> : alone
<UTe> : rate
<IvY> : separate
<NKz> : SCARLET
<XYo> : O
<xNY> : minor
<QGU> : competition
<iKT> : ABIGAIL
<fFy> : woman
<imI> : adjacent
<mCH> : grapple
<CsO> : evaluating
<Uqw> : SMITH
<xma> : C
<CKl> : trailing
<eAd> : opening
<GbF> : form
<OAl> : while
<oOn> : STR
<TLY> : blocks
<geg> : compressed
<uNb> : Fixed
<RrM> : symbols
<IWm> : asymptotic
<bBV> : a
<Juw> : cashback
<NVz> : The
<cVn> : alphabetical
<HvW> : them
<aWm> : if
<NuY> : f
<Ezq> : flat
<UGi> : middle
<QRw> : problems
<qbO> : replace
<zJB> : do
<fJN> : real
<YUh> : models
<IYT> : Exponents
<iZC> : function
<MGS> : Alliant
<kJh> : only
<KFn> : 7
<giX> : `
<EVn> : computer
<MyM> : BLAKE
<zJH> : :
<eNb> : established
<YGN> : like
<vuQ> : OF
<mqp> : One
<PaC> : /
<TAl> : language
<nat> : tables
<RsY> : returns
```

```
<sSU> : open
<dJg> : Big
<dph> : (
<GFv> : both
<ptB> : for
<BFP> : supporting
<gjf> : Let
<cCu> : F
<TeL> : space
<aLs> : problem
<vBU> : related
<WEU> : given
<bNc> : information
<DNh> : base
<dCh> : first
<RmM> : different
<Zgt> : person
<tqc> : ,
<rxl> : album
<kMm> : when
<CiP> : their
<UvZ> : immediately
<huu> : 0
<qhY> : FRIENDSHIP
<WiY> : Identify
<OEo> : array
<Dmi> : Lamar
<UtQ> : represents
<oVz> : length
<dzf> : least
<Uqo> : For
<acI> : this
<jLk> : rather
<oVM> : table
<Uee> : edge
<Wyo> : possibly
<Xdq> : Include
<soY> : return
<OgM> : achievable
<RwJ> : exists
<xpP> : far
<Fam> : ”
<tUs> : allowed
<PKI> : slots
<hTv> : must
<LCV> : character
<uoI> : produced
<StI> : H
<DTM> : optimal
<vRQ> : variable
<aCx> : perk
<QKU> : Kendrick
<cGa> : tools
<ICY> : associated
<nyv> : Qwen
<Oou> : 5
<xNw> : Free
<pLA> : ranking
```

```
<Tor> : written
<Qzc> : tight
<YjI> : format
<Fiu> : effect
<ERW> : letters
<zgn> : date
<lTA> : fixed
<Isj> : concept
<ShZ> : called
<QFo> : gun
<GGe> : define
<TVi> : require
<Bcf> : _
<KHO> : variables
<BEO> : th
<wUr> : every
<GOm> : same
<fmU> : applying
<KXI> : obtained
<din> : include
<oHZ> : 4
<QIb> : at
<jcm> : goes
<VSZ> : JONES
<Qax> : never
<pvy> : meaning
<cLv> : bit
<cxi> : zombies
<Wki> : operations
<ZxL> : plots
<SIk> : letter
<UyT> : pair
<gsj> : parenthesis
<yPU> : computed
<SVV> : claymation
<lHq> : D
<Cqd> : on
<VaB> : E
<rdm> : characters
<tBO> : uses
<OhH> : using
<diL> : Format
<Rjf> : cult
<BFL> : around
<TqF> : meaningful
<kLB> : It
<xfF> : bounds
<RTV> : v
<Icm> : follow
<uSp> : song
<nGQ> : that
<iFh> : encodes
<HQX> : as
<FRD> : references
<Wkl> : than
<Bti> : answer
<TgU> : match
<QMt> : contains
```

```
  <yXQ> : you
  <IqZ> : right
  <ctS> : simultaneously
  <MhO> : appear
  <oaf> : part
  <MZy> : Trim
  <atx> : y
  <Iwb> : positive
  <gvZ> : A
  <IOe> : multiple
  <mYI> : purchases
  <uZj> : block
  <Ods> : LLM
  <Ctn> : have
  <vvS> : paper
  <RHH> : separated
  <YfC> : valid
  <LVD> : people
  <Qdp> : terms
  <POy> : YYYYMMDD
  <Umn> : ZACHARY
  <Llz> : ASCII
  <YTt> : unchanged
  <lNZ> : nlog
  <ufF> : additive
  <Kws> : n
  <yGF> : directly
  <zAC> : Releases
  <NDo> : bound
  <sey> : Theta
  <ajQ> : theoretical
  <ZUw> : x
  <kkG> : composite
  <cOS> : factor
  <BGb> : sequence
  <jQO> : construction
  <auW> : B
  <ETe> : discontinuation
  <foF> : all
  <YNh> : Because
  <ebp> : 2
  <jcU> : underscore
  <BQY> : rules
  <jzP> : )
  <mYs> : appears
  <yuL> : above
  <udh> : an
  <SMe> : pointer
  <iDq> : vertex
  <hPR> : science
  <YJu> : will
  <sIC> : optional
  <Umv> : replaced
  <ZOV> : As
  <xut> : edges
  <xdv> : Every
}
```

Do what that prompt says and follow its instructions. The prompt is below:

<gvZ> <KFi> <ptB> <CsO> <TAl> <YUh> <Cqd> <EKS> <QGU> <QRw><tqc> <ndx><tqc> <psp> <bBV> <vvS> <ICY>
<pGz> <uVw><nsH> <NVz> <vvS> <QMt> <QIb> <dzf> <NuI><tqc> <OHv> <Wyo> <IOe> <pLA> <nat><nsH> <mqp> <oVM>
<mYs> <yGF> <yuL> <bBV> <ArI> <NtB> <nGQ> <ZxL> <LLc> <ptB> <wPo><oHZ><gxy><prh> <MkU>
<nyv><eNa><gxy><eNa><huu><auW><gxy><gvZ><eNa><auW><tqc> <gzB> <JVi> <YUh><nsH>

<fUP> <nGQ> <QXS> <oVM><tqc> <mIH> <oPz> <QMt> <bBV> <loP> <zcW> <LQz> <PzH> <bNc> <rlx> <ICk><nsH>
<QwG> <RCG> <FsZ> <Let> <nGQ> <mYs> <UvZ> <NYA> <FsZ> <Qgf> <TyO> <FsZ> <dCh> <eAd> <gsj> <rlx> <FsZ>
<oHZ><BEO> <oPz> <TyO> <nGQ> <oVM><lYD> <MZy> <MhQ> <MWr> <uMR> <CKl> <VPu> <xSZ> <qyv> <Bti><nsH> <pmf>
<qyv> <Bti> <rlx> <foF> <gOg> <MkU> <qbO> <MhQ> <VPu> <pGz> <HSs><nsH>

<gjf> <gvZ><Bcf><oOn> <gVD> <nGQ> <Let><nsH>

<pzd><zJH> <Fam><vSu><Bcf><ebp><nsH><huu><Bcf><IZD><Fam>

<fUP> <NtB> <yfW><tqc> <FsZ> <Exu> <blh> <TyO> <bBV> <NtB> <yBr> <QIb> <bBV> <iDq> <RTV> <RCG> <FsZ>
<pPL> <nGQ> <iQH> <wUr> <Uee> <Ydj> <sPb> <rCL> <TyO> <RTV><tqc> <ZEv> <FsZ> <ONV> <AgK> <Cqd> <FsZ>
<sSU> <vta> <sAJ><dph><RTV><jzP> <pGz> <PXQ> <pbv><tqc> <OAl> <JJz> <foF> <xut> <SxB> <Ydj> <khZ> <rCL>
<TyO> <RTV> <YTt><nsH>

<gvZ> <NtB> <StI> <RCG> <bBV> <iDq><gxy><xNY> <TyO> <bBV> <NtB> <yBr> <aWm> <StI> <DJz> <gVD> <KXI> <xSZ>
<yBr> <LQz> <dCh> <Wny> <udh> <ONV> <AgK> <MkU> <tqN> <fmU> <bBV> <BGZ> <BGb> <TyO> <Exu> <RAy><nsH>

<Uqo> <bBV> <Iwb> <dVt> <MAv><tqc> <GGe> <NuY><dph><MAv><jzP> <HQX> <FsZ> <uKX> <dVt> <Kws> <Gea> <nGQ>
<wUr> <IVg> <NtB> <Cqd> <mWT> <Kws> <FuI> <QMt> <FsZ> <NCG> <NtB> <Cqd> <MAv> <FuI> <dph><MAv> <uol>
<KQJ><gxy><imI> <FuI> <pGz> <ExV> <xut> <gzB> <HvW><jzP> <HQX> <bBV> <iDq><gxy><xNY><nsH>

<kLB> <RCG> <oIO> <nGQ> <NuY><dph><MAv><jzP> <zaL> <ebp><eOo><MAv> <gxy> <tiT> <ptB> <MAv> <zaL>
<tiT><tqc> <ebp><tqc> <MkU> <eNa><nsH>

<QwG> <RCG> <NuY><dph><oHZ><jzP><lYD> <gjf> <auW><Bcf><oOn> <gVD> <nGQ> <dVt><nsH>

<pzd><zJH> <Fam><KFn><Fam>

<fUP> <bBV> <oYW> <jLN> <TyO> <bBV> <uSp> <LQz> <QKU> <Dmi><tqc> <FsZ> <cIt> <TyO> <khZ> <RmM> <fJN>
<LVD> <EyO> <EaR> <oHp> <vMH> <NuI> <kkG> <Kfk><nsH> <WiY> <GFv> <LVD> <MkU> <soY> <CiP> <BEI> <wAY>
<cIt> <dph><aXy> <foF> <WEU> <cIt> <MkU> <MhQ> <UGi> <cIt><jzP><nsH> <NGH> <GFv> <LVD><bcz><sqL> <BEI>
<wAY> <cIt> <rlx> <cVn> <YZh> <LQz> <avo> <Kfk><tqc> <RHH> <LQz> <bBV> <oYW> <jcU><nsH> <NDA> <udh> <jcU>
<xWC> <yXQ> <Kgy> <pUM> <bBV> <TeL> <rlx> <CiP> <cIt><nsH> <pmf> <qyv> <Bti> <rlx> <foF> <gOg><nsH>

<gjf> <xma><Bcf><oOn> <gVD> <nGQ> <Let><nsH>

<pzd><zJH> <Fam><iKT><Bcf><BmK><Bcf><VSZ><Bcf><Umn><Bcf><MyM><Bcf><Uqw><Fam>

<QwG> <fFy> <RCG> <iJY> <NYA> <gVD> <bBV> <oaf> <TyO> <bBV> <QKU> <rxl><tqc> <RwJ> <HQX> <bBV> <Isj>
<jLk> <Wkl> <bBV> <GyP> <Zgt><tqc> <MkU> <tBO> <bBV> <QFo><lYD> <pmf> <qyv> <Bti> <rlx> <foF> <gOg><nsH>
<WYc> <MhQ> <VPu> <pGz> <HSs><nsH>

<gjf> <lHq><Bcf><oOn> <gVD> <nGQ> <Let><nsH>

<pzd><zJH> <Fam><NKz><Bcf><DVW><Fam>

<gvZ> <TtM><gxy><oVM> <RCG> <bBV> <UbW> <oWc> <pGz> <Kws> <PKI> <BFP> <HLO> <Wki><zJH>

<gxy> <QXh><dph><UyH><jzP> <gxy><YXK> <jOr><zJH> <Djk> <bBV> <gmj> <TtM> <ptB> <giX><UyH><giX> <MkU>
<RsY> <bBV> <geg> <KdF> <giX><jOr><giX><nsH>

```
<gxy> <WUk><dph><UyH><tqc> <jOr><jzP> <gxy><YXK> <TtM><Bcf><mRY><zJH> <NAO> <FsZ> <mRY> <TyO> <FsZ>
<TtM> <kzX> <LQz> <giX><UyH><giX><nsH> <dQN> <YfC> <jfp> <ShZ> <pGz> <FsZ> <GOm> <UyH> <nGQ> <uoI>
<giX><jOr><giX><nsH>
<gxy> <xNw><dph><UyH><tqc> <jOr><jzP><zJH> <zAC> <FsZ> <TtM><nsH>

<YNh> <giX><jOr><giX> <RCG> <kJh> <TqF> <oHp> <pGz> <PXQ> <ZrT> <giX><UyH><giX><tqc> <FsZ> <UyT>
<dph><UyH><tqc> <jOr><jzP> <iFh> <FsZ> <TtM> <mRY> <jLk> <Wkl> <giX><jOr><giX> <gba><nsH> <Bye> <FJU>
<giX><jOr><giX> <NYA> <pUM> <xpP> <GEv> <iID> <Wkl> <FsZ> <JYw><dph><Kws><jzP><gxy><cLv> <eVK> <NDo>
<nGQ> <Kgy> <MOA> <NYA> <bBV> <JuZ> <XMh> <SMe> <vMH> <udh> <OEo> <TyO> <gxb> <Kws><nsH>

<NVz> <oVM> <ACh> <QIb> <RNF> <cOS> <tiT> <gxy> <tiT><PaC><MAv><tqc> <pvy> <QIb> <Tek> <Kws><dph><tiT>
<gxy> <tiT><PaC><MAv><jzP> <PKI> <DJz> <gVD> <ctS> <SMb><nsH>

<uNb><gxy><gxb> <FRD> <TVi> <wUr> <giX><jOr><giX> <NYA> <pUM> <FsZ> <GOm> <uVe> <TyO> <iID><nsH> <gvZ>
<ebp><huu><ebp><tiT> <ajQ> <EVn> <hPR> <Zmp> <eNb> <Qzc> <IWm> <xfF> <gxy><gxy> <bBV> <rKT> <jQO> <MkU>
<bBV> <eVK> <NDo> <gxy><gxy> <Cqd> <FsZ> <cMt> <OgM> <lTA><gxy><gxb> <KdF> <oVz><nsH>

<QwG> <RCG> <FsZ> <Vnv> <Qzc> <NDo> <rlx> <sey><gxy><xiW> <Cqd> <FsZ> <DTM> <lTA><gxy><gxb> <KdF> <gxb>
<rlx> <iID><tqc> <HQX> <bBV> <iZC> <TyO> <Kws> <MkU> <MAv><lYD>

<Qvu> <YjI> <BQY> <ptB> <VaB><Bcf><oOn> <dph><Icm> <mWT> <uMR> <qyv> <Bti> <YJu> <SxB> <TgU><jzP><zJH>
<gxy> <NDA> <FsZ> <vBt> <LCV> <XYo> <dph><SxB> <Fam><sey><Fam> <uMR> <Fam><dJg><gxy><sey><Fam><jzP>
<gxy> <xdv> <JYw> <hTv> <Ctn> <ICk> <BFL> <PXQ> <OpO><zJH> <JYw><dph><ZUw><jzP><tqc> <Qax> <JYw> <ZUw>
<gxy> <BCG> <zJB> <SxB> <KNh> <NYA> <axw> <FsZ> <DNh> <TyO> <MhQ> <JYw>
<gxy> <IYT> <pUM> <FsZ> <GbF> <ZUw><eOo><dph><atx><jzP> <gxy><gxy> <ICk> <EyO> <KQJ><gxy><sIC>
<gxy> <KkX> <bBV> <LHZ> <QMt> <GFv> <bBV> <vRQ> <MkU> <bBV> <JYw> <TyO> <nGQ> <vRQ><tqc> <FsZ> <vRQ>
<jcm> <NYA> <FsZ> <Qgf> <TyO> <FsZ> <JYw><zJH> <HKo><dph><ZUw><jzP><tqc> <SxB> <JYw><dph><ZUw><jzP><ZUw>
<gxy> <KkX> <qyv> <Bti> <psp> <IOe> <ufF> <Qdp><tqc> <YZh> <HvW> <Qgf> <NYA> <IqZ> <xSZ> <Tek> <rdm>
<NYA> <Asr> <rdm>
<gxy> <cyr> <NYA> <FsZ> <Qgf> <MkU> <IqZ> <TyO> <TPx> <MkU> <gxy> <RrM>

<gjf> <VaB><Bcf><oOn> <gVD> <FsZ> <sey> <phr> <mWT> <HQX> <Tor><nsH>

<pzd><zJH> <Fam><XYo><dph><lNZ><dph><Kws><jzP> <TPx> <Kws><eOo><dph><ebp><jzP><jzP><Fam>

<NVz> <MGS> <qkv> <PgQ> <RHy> <tEy> <ebp><nsH><Oou><HSI> <Ezq><gxy><UTe> <Juw> <Cqd> <mYI><nsH> <QwG>
<RCG> <FsZ> <Efc> <zgn> <nGQ> <FsZ> <ETe> <TyO> <acI> <aCx> <iHw> <Fiu><lYD> <diL> <qyv> <Bti> <HQX>
<POy> <pGz> <ExV> <SqB> <uMR> <JVi> <eKu><nsH>

<gjf> <cCu><Bcf><oOn> <gVD> <nGQ> <zgn> <Let><nsH>

<pzd><zJH> <Fam><ebp><huu><ebp><oHZ><huu><tiT><huu><tiT><Fam>

<XEE> <rlx> <FsZ> <AFn><zJH>

<Bcf> <Bcf><Bcf><Bcf><Bcf><Bcf> <Bcf><Bcf><Bcf><Bcf> <Bcf><Bcf><Bcf><Bcf><Bcf><Bcf><Bcf><Bcf><Bcf><Bcf>

<TfK> <jcU> <UtQ> <NuI> <SIk> <MkU> <VPu> <IvY> <aaa><nsH> <NVz> <Bti> <RCG> <VBP> <aaa> <pGz>
<tiT><tqc> <Oou><tqc> <oHZ><tqc> <MkU> <tiT><huu> <ERW> <KkM><nsH>

<NVz> <Bti> <RCG> <Cjx> <vBU> <NYA> <foF> <TyO> <FsZ> <IrP><zJH> <Rjf><tqc> <QFo><tqc> <APy><tqc> <mCH>
<nsB><tqc> <bRh><tqc> <FVQ><tqc> <SVV><tqc> <cxi><nsH>

<pmf> <qyv> <Bti> <rlx> <foF> <gOg> <MkU> <qbO> <MhQ> <VPu> <pGz> <HSs><nsH>

<gjf> <yBr><Bcf><oOn> <gVD> <nGQ> <Let><nsH>

<pzd><zJH> <Fam><pRz><Bcf><yWI><Bcf><vuQ><Bcf><qhY><Fam>
```

```
<ZOV> <udh> <Ods><tqc> <yXQ> <EyO> <tUs> <NYA> <pUM> <MhQ> <cGa><tqc> <vMk><tqc> <uMR> <JVi> <yTY> <QIb>
<qyv> <Cbr> <NYA> <pQB> <acI> <aLs><nsH>

<pmf> <qyv> <Bti> <rlx> <bBV> <cwu> <uZj> <YGN> <acI><zJH>
<giX><giX><giX>
<gvZ><Bcf><oOn><gxy><auW><Bcf><oOn><gxy><xma><Bcf><oOn><gxy><lHq><Bcf><oOn><gxy><VaB><Bcf><oOn><gxy><cC
u><Bcf><oOn><gxy><yBr><Bcf><oOn>
<giX><giX><giX>

<pzd><zJH> <Fam><vSu><Bcf><ebp><nsH><huu><Bcf><IZD><gxy><KFn><ebp><eNa><gxy><iKT><Bcf><BmK><Bcf><VSZ><B
cf><Umn><Bcf><MyM><Bcf><Uqw><gxy><NKz><Bcf><DVW><gxy><XYo><dph><lNZ><dph><Kws><jzP> <TPx>
<Kws><eOo><dph><ebp><jzP><jzP><gxy><ebp><huu><ebp><oHZ><huu><tiT><huu><tiT><gxy><pRz><Bcf><yWI><Bcf><vuQ
><Bcf><qhY><Fam>

<Uqo> <Let> <KHO> <gvZ><tqc> <xma><tqc> <lHq><tqc> <MkU> <yBr><zJH> <MhQ> <KQJ> <foF><gxy><gOg> <SIk>
<gUq> <gVD> <Umv> <pGz> <udh> <jcU><nsH> <Xdq> <auW><Bcf><oOn><tqc> <VaB><Bcf><oOn><tqc> <MkU>
<cCu><Bcf><oOn> <mWT> <HQX> <yPU> <pGz> <ExV> <DuY><nsH>

<lrV> <acI> <aLs> <kMm> <OhH> <FJH><gxy><XYo><tqc> <uVw> <ZRs> <FJH><gxy><gxz><nsH> <mds>
<KQJ><gxy><Llz> <rdm> <gUq> <MhO> <rlx> <qyv> <Bti><nsH>

<Nnb> <SxB> <din> <MhQ> <JVi> <cwu> <TLY> <rlx> <qyv> <SXl><nsH>
```